DLP Assignment 2

Question 1

A)

1. large 11×11 convolution layer → AlexNet
2. repeated 3×3 convolution → VGGn
3. map is increased while spatial resolution is reduced → CNN downsampling
4. Shortcut connection → resNet
5. different kernel sizes → Inception
6. global avg. pooling before classification → Inception & ResNet

B)

1- AlexNet :- training deep CNNs on large datasets was computationally expensive. AlexNet used large strides and max pooling to reduce image size quickly, while ReLU, dropout, and GPUs helped speed up training and reduce overfitting.

2- VGGn :- large convolution filters require parameters and computations. VGG used small 3×3 filters, allowing deeper networks to learn complex features by adding more ReLUs and using fewer parameters.

4- ResNet :- very deep networks become difficult to train because gradients can become too small and adding more layers may reduce accuracy. Shortcut connections adds the input straight to the output of a few layers so it helps information and gradients to flow through the network more easily.

Scanned with CamScanner

5- Inception :- objects can appear in different sizes, so a simple filter size may not capture all object features. parallel branches with different kernel sizes allow the network to detect both small and large patterns.

6- Large fully connected layers contain many parameters, increasing computation and the risk of overfitting. Global avg. pooling averages each feature map into a single value, reducing parameters and simplifying classifications.

D) A plain network has gradients that shrink as they travel back through many layers so it ~~d~~trains badly. A shortcut network has gradients that flow straight through the skip path. Each layer only has to learn a small correction, so ~~d~~training is much easier.

E) It can provide architectural advantages because multiple 3×3 filters can cover the same area as a 7×7 filter with few parameters and learns more complex features through extra layers. Inception simply doesn't replace every operation because it doesn't use only 3×3 filters because objects have different sizes. It uses different filter sizes in parallel to capture different features. and 1×1 filters to reduce computation.

Question 2

Input = 227 × 227 × 3

A.

No. of parameters in convolutional layer = $(K_h \times K_w \times C_{in} \times C_{out}) + C_{out}$

K_h : kernel height, K_w : kernel width, C_{in} : input channels, C_{out} : no. of filters, C_{out} : bias of each filter

1. conv 1 = 11×11 kernel, 96 filters

parameters = $(11 \times 11 \times 3 \times 96) + 96 = 34\,944$

2. conv 2 = 5×5, 256

parameters = $(5 \times 5 \times 96 \times 256) + 256 = 614\,656$

3. conv 3 = 3×3, 384

parameters = $(3 \times 3 \times 256 \times 384) + 384 = 885\,120$

Scanned with CamScanner

4. conv 4 = 3×3, 384

Parameters = $(3 \times 3 \times 384 \times 384) + 384 = 1327488$

5. conv 5 = 3×3, 256

Parameters = $(3 \times 3 \times 384 \times 256) + 256 = 884992$

B. output size of a conv layer = $\left[\frac{N + 2p - k}{S}\right] + 1$

conv 1 = $\left[\frac{227 + 2(0) - 11}{4}\right] + 1 \Rightarrow 55 \times 55 \times 96$

pool conv 2 = $\left[\frac{55 + 2(0) - 3}{2}\right] + 1 \Rightarrow 27 \times 27 \times 96$

conv 2 = $\left[\frac{27 + 4 - 5}{1}\right] + 1 = 27 \times 27 \times 256$

pool 2 = $\left[\frac{27 - 3}{2}\right] + 1 = 13 \times 13 \times 256$

conv 3 = $\left[\frac{13 + 2 - 3}{1}\right] + 1 = 13 \times 13 \times 384$

conv 4 = $\left[\frac{13 + 2 - 3}{1}\right] + 1 = 13 \times 13 \times 384$

conv 5 = $\left[\frac{13 + 2 - 3}{1}\right] + 1 = 13 \times 13 \times 256$

pool 3 = $\left[\frac{13 - 3}{2}\right] + 1 = 6 \times 6 \times 256$

Input to FC1 = flattened vector of $6 \times 6 \times 256 \Rightarrow$ 9216 input values

C. parameters in FC1, FC2, FC3

parameters = $(N_{input} \times N_{out}) + N_{out}$

FC1 = input 9216, 4096 neurons

$P = (9216 \times 4096) + 4096 = 37752832$

FC2 = input 4096, 4096 neurons

$P = (4096 \times 4096) + 4096 = 16781312$

FC3 = input 4096, 1000 neurons

$P = (4096 \times 1000) + 1000 = 4097000$

Scanned with CamScanner

D. Total no. of trainable parameters

$\Rightarrow$ 34944 + 64656 + 885120 + 1327488 + 884992 + 37752832 + 16781312 + 4097000 = 62378344 parameters

Sum of all convolutional layers and fully connected layers

Question 3

Flow :- $32 \times 32 \times 1 \rightarrow$ conv $(5\times5)\times6 = 28\times28\times6 \rightarrow$ avg pool $= 14\times14\times6$ $\rightarrow$ conv $= 10\times10\times16 \rightarrow$ avg pool $= 5\times5\times16 = 400 \rightarrow$ FC120 $\rightarrow$ FC84 $\rightarrow$ Softmax 10

1) conv 1 $\Rightarrow (5\times5\times1\times6) + 6 = 156$, avg pooling = no trainable parameters

conv2 $\Rightarrow (5\times5\times6\times16) + 16 = 2416$, avg pooling = no trainable parameters

2) FC1 $\Rightarrow (400\times120) + 120 = 48120$

FC2 $\Rightarrow (120\times84) + 84 = 10164$

3) softmax with 10 output neurons = $(84\times10) + 10 = 850$

4) Total parameters = 156 + 2416 + 48120 + 10164 + 850 = 61706 trainable parameters

Scanned with CamScanner